

¿Qué es un formulario contenedor (MDI)?

Una interfaz de documentos múltiples (MDI, por sus siglas en inglés Multiple Document Interface) es un patrón de diseño de interfaz gráfica de usuario en el que múltiples ventanas secundarias (documentos o formularios hijos) se encuentran contenidas dentro de una única ventana principal (ventana padre o contenedor).

En el contexto de las aplicaciones de escritorio, permite que el usuario trabaje con varios documentos o formularios simultáneamente sin salir de la aplicación principal. La ventana principal actúa como un escritorio que gestiona la visualización, el tamaño y la posición de las subventanas, las cuales no pueden salir de los límites del contenedor principal.

¿Qué es un JDesktopPane?

JDesktopPane es un componente de la biblioteca Java Swing (`javax.swing.JDesktopPane`) que actúa como el contenedor principal o el "escritorio virtual" para las aplicaciones MDI.

Funciona como una subclase de `JLayeredPane` diseñada específicamente para albergar y gestionar múltiples instancias de `JInternalFrame`. El `JDesktopPane` se encarga de administrar el orden de las capas (z-order) de las ventanas hijas, gestionando cuál ventana está al frente (enfocada), cuál está minimizada, maximizada o apilada.

¿Qué es un JInternalFrame?

`JInternalFrame` es un componente de Java Swing (`javax.swing.JInternalFrame`) que simula una ventana o formulario independiente, pero con la restricción de que solo puede existir y mostrarse dentro de un `JDesktopPane`.

Este componente hereda la mayor parte de su comportamiento de `JFrame`, lo que significa que posee una barra de título, botones de minimizar, maximizar, cerrar, un borde redimensionable y la capacidad de contener menús y componentes gráficos (botones, tablas, campos de texto, etc.). Permite al desarrollador crear la sensación de múltiples ventanas independientes dentro de una sola ventana contenedora.

¿Cómo funciona un JMenuBar?

JMenuBar (javax.swing.JMenuBar) es la clase de Swing utilizada para implementar una barra de menús horizontal estándar en una ventana de aplicación.

Funcionamiento:

1. **Contenedor de Menús:** La barra de menús se añade a la ventana principal (JFrame) mediante el método `setJMenuBar(barraDeMenu)`. Contiene objetos de tipo JMenu (como "Archivo", "Editar", "Ventana").
2. **Desplegables:** Al hacer clic o pasar el cursor sobre un JMenu, este despliega una lista de opciones representadas por objetos JMenuItem.
3. **Gestión de Eventos:** A cada JMenuItem se le añade un escuchador de eventos (ActionListener) que detecta cuando el usuario hace clic en la opción.
4. **Interacción MDI:** En una arquitectura MDI, los elementos del menú se utilizan comúnmente para instanciar, mostrar, minimizar o cerrar los formularios hijos (JInternalFrame) dentro del JDesktopPane.
- 5.

¿Cómo abrir formularios hijos desde un menú?

Para abrir un formulario hijo (JInternalFrame) desde una opción de menú (JMenuItem), se sigue un procedimiento estándar basado en programación orientada a eventos:

1. **Crear la clase del formulario hijo:** Diseñar una clase que extienda de JInternalFrame con sus componentes internos.
2. **Vincular el ActionListener:** Agregar un escuchador de eventos al JMenuItem deseado.
3. **Instanciar y configurar:** Dentro del evento, se crea una instancia del formulario hijo, se configuran sus propiedades (tamaño, visibilidad, si es redimensionable, etc.).
4. **Agregar al DesktopPane:** Se añade la instancia creada al JDesktopPane principal mediante el método `add()`.
5. **Mostrar y enfocar:** Se invoca el método `setVisible(true)` y opcionalmente `setSelected(true)` para traer la ventana al frente.

Ejemplo:

```
7 import javax.swing.*;
8 import java.awt.event.ActionEvent;
9 import java.awt.event.ActionListener;
10
11 public class VentanaPrincipalMDI extends JFrame {
12     private JDesktopPane desktopPane;
13
14     public VentanaPrincipalMDI() {
15         setTitle("Ejemplo Arquitectura MDI");
16         setSize(800, 600);
17         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
18
19         // 1. Inicializar el JDesktopPane
20         desktopPane = new JDesktopPane();
21         add(desktopPane);
22
23         // 2. Crear la barra de menús
24         JMenuBar menuBar = new JMenuBar();
25         JMenu menuArchivo = new JMenu("Archivo");
26         JMenuItem itemAbrir = new JMenuItem("Abrir Formulario Hijo");
27
28         // 3. Programar la acción de abrir el formulario hijo desde el menú
29         itemAbrir.addActionListener(new ActionListener() {
30             @Override
31             public void actionPerformed(ActionEvent e) {
32                 crearYMostrarFormularioHijo();
33             }
34         });
35
36         menuArchivo.add(itemAbrir);
37         menuBar.add(menuArchivo);
38         setJMenuBar(menuBar);
39     }
40
41     private void crearYMostrarFormularioHijo() {
42         // Crear instancia del formulario hijo (JInternalFrame)
43         JInternalFrame formularioHijo = new JInternalFrame("Formulario Interno", true, true, true, true);
44         formularioHijo.setSize(300, 200);
45         formularioHijo.setLocation(50, 50);
46
47         // Añadir componentes al formulario hijo si es necesario
48         JPanel panel = new JPanel();
49         panel.add(new JLabel("¡Hola, soy un formulario hijo!"));
50         formularioHijo.add(panel);
51
52         // Añadir el formulario al JDesktopPane y hacerlo visible
53         desktopPane.add(formularioHijo);
54         formularioHijo.setVisible(true);
55     }
56
57     public static void main(String[] args) {
58         SwingUtilities.invokeLater(() -> {
59             new VentanaPrincipalMDI().setVisible(true);
60         });
61     }
62 }
```

Ventajas:

- **Organización del espacio de trabajo:** Mantiene todas las ventanas relacionadas agrupadas dentro de una sola ventana principal de la aplicación, evitando el desorden en el escritorio del sistema operativo.
- **Gestión centralizada de recursos:** Facilita la administración de barras de herramientas, menús globales y estados de la aplicación desde una única interfaz padre.
- **Experiencia de usuario fluida:** Permite al usuario alternar rápidamente entre múltiples documentos, vistas o formularios sin perder el contexto de la aplicación principal.
- **Control de ciclo de vida:** El contenedor principal (JDesktopPane) simplifica la tarea de controlar qué ventanas están abiertas, minimizadas o cerradas simultáneamente
- **Independencia visual:** Los formularios hijos operan de manera autónoma en su interior, permitiendo redimensionamiento, minimización y cierre individual sin afectar al resto del sistema.

Bibliografía o fuentes consultadas

- Oracle. (s. f.). *How to Use Internal Frames*. Java Tutorials - Oracle Documentation. Recuperado de la documentación oficial de Java Swing.
- Oracle. (s. f.). *Class JDesktopPane*. Java Platform SE. Recuperado de la especificación de la API de Java.
- Oracle. (s. f.). *Class JInternalFrame*. Java Platform SE. Recuperado de la especificación de la API de Java.
- Deitel, H. M., & Deitel, P. J. (2012). *Java: How to Program* (9.^a ed.). Prentice Hall.